# Programlama Dilleri

Dr. Öğr. Üyesi Hayri Volkan Agun

Bilgisayar Mühendisliği Bölümü

Bursa Teknik Üniversitesi

# Programlama Dilleri - Soyutlama

- Programlama dilleri zamanla **fiziksel makineden uzaklaşmıştır**
- Bu evrim, **soyutlama mekanizmaları** sayesinde gerçekleşmiştir
- Her yeni seviyede:
- Düşük seviyeli detaylar gizlenir
- Daha güçlü ifade imkânı sağlanır

- Soyutlama sayesinde:
    - **Veri**, bellek düzeninden bağımsız temsil edilir
    - **Kontrol akışı**, makine talimatları olmadan ifade edilir
- Temel fikir: Somut makine detaylarını **sembolik temsillerle değiştirmek**

# İsimleme

- Bir karakter dizisidir
- Başka bir varlığı temsil eder
- Örnekler:
  - Değişkenler → bellek konumları
  - Fonksiyonlar → işlem dizileri
- İsimler, **soyutlamanın temelidir.**

# İsimleme

- Veri Soyutlaması
  - Bir isim, **bir bellek konumunu veya değeri temsil eder**
  - Örnek:
    - int x = 10;
    - → x bir bellek hücresini temsil eder

- Bir isim, bir komut kümesini temsil eder.
  - Örnek:def compute(): ...
  - → compute bir prosedürü temsil eder

# İsimleme

- Açık gibi görünse de önemli bir nokta:
  - Bir isim ile temsil ettiği nesne aynı şey değildir.
  - İsim:Sadece bir karakter dizisidir.
  - Gösterdiği (denotation):Değişken, fonksiyon, tip gibi karmaşık yapılar olabilir.
- Bir nesnenin: Birden fazla ismi olabilir → aliasing (takma adlama)
- Bir ismin:Farklı zamanlarda farklı nesneleri göstermesi mümkündür.
  - Bu yüzden:“değişken fie” ≈ “ismi fie olan değişken”
  - “fonksiyon fie” ≈ “ismi fie olan fonksiyon”
  - Pratikte:Bir isim kullanıldığında, genellikle gösterdiği nesne kastedilir

# Gösterilebilir Nesneler

- Bir ismin temsil edebildiği nesnelere *Gösterilebilir Nesneler* (Denotable Objects) denir.
- Programlama dillerine göre değişse de, kapsam oldukça geniştir.
- Kullanıcı tarafından isim verilen nesneler:
  - Değişkenler (variables)
  - Biçimsel parametreler (formal parameters)
  - Prosedürler / fonksiyonlar
  - Kullanıcı tanımlı tipler
  - Etiketler (labels)
  - Modüller
  - Sabitler (constants)
  - İstisnalar (exceptions)

# Bağlama (Binding)

- Binding (bağlama):
  - Bir isim ile onun temsil ettiği nesne arasındaki ilişkidir.
  - Bu ilişki farklı zamanlarda oluşturulabilir.

```
{int fie;
 fie = 2;
    {char fie;
     fie = a;
    }
}
```

- Programlama süreci boyunca bağlama farklı aşamalarda gerçekleşir:
  - Dil tasarımı
  - Program yazımı
  - Derleme zamanı
  - Çalışma zamanı (runtime)

# Bağlama (Binding)

- Program yazım zamanı.
  - Programcı:İsimleri seçer.
  - Bazı bağlamaları tanımlar
  - Ancak:Bu bağlamalar tam olarak henüz oluşmaz
    - 📌 Örnek:
      - Bir değişken tanımlanır.
      - Bellekte yer ayrılması henüz yapılmamıştır.

- Derleme Zamanı (Compile Time).
  - Derleyici:
    - Yüksek seviyeli kodu makine koduna çevirir
    - Statik olarak işlenebilen veriler için bellek ayırır
  - 📌 Örnek:
    - Global değişkenler
    - Bu aşamada:
      - İsim ↔ bellek adresi ilişkisi kurulur

# Bağlama (Statik ve Dinamik)

- Çalışma Zamanı (Runtime)
  - Programın çalıştığı süre:
  - Başlangıç → bitiş arası
  - Bu aşamada:Önceden oluşturulmayan tüm bağlamalar kurulur.
  - 📌 Örnekler:
    - Rekürsif fonksiyonlardaki yerel değişkenler
    - Dinamik bellek (pointer, heap allocation)

- Statik vs Dinamik Bağlama
  - Programlama dillerinde iki temel ayrım:
  - Statik (Static)
    - Program çalışmadan önce gerçekleşir
      - Derleme zamanı işlemleri
      - Statik bellek yönetimi
  - Dinamik (Dynamic)
    - Program çalışırken gerçekleşir
      - Runtime bellek yönetimi
      - Dinamik bağlama

# Ortam (Environment)

- Tüm isim–nesne ilişkileri program başında sabit değildir.
- Birçoğu çalışma sırasında değişebilir.
- 👉 Bu davranışı anlamak için ortam (Environment) kavramı gereklidir.
- Ortam programın belirli bir noktasında ve çalışma anında mevcut olan isimler ile gösterdikleri nesneler arasındaki ilişkiler kümesi
- Yani:“Bu isim şu anda neyi temsil ediyor?” sorusunun cevabını verir.
- Ortamın rolü genellikle sadece programcı tarafından tanımlanan isimleri kapsar
- Ortam:Her program noktasında
- Her isim için doğru eşleşmeyi belirler.
- 📌 Not:Ortam Fiziksel makinede doğrudan yoktur. Soyut makinenin bir bileşenidir.

# Bildirim (Decleration)

```
int x;
int f() {
  return 0;
}
type T = int;
```

- x → değişken bildirimi
- f → fonksiyon bildirimi
- T → yeni tip bildirimi

Açık ve Örtük Bildirimler

- Açık Bildirim (Explicit): Programcı açıkça tanımlar. Örnek: int x;
- Örtük Bildirim (Implicit)İlk kullanımda oluşur. Tür bağlamdan çıkarılır.

Aynı Nesne, Farklı İsimler

- Bir nesne farklı ortamlarda birden fazla isimle erişilebilir
- 📌 Örnek:
  - Referans ile parametre geçirme.
  - Pointer kullanımı

# Aliasing

- Aynı nesnenin:aynı ortamda birden fazla isimle erişilmesi
- Bu isimlere:alias (takma ad) denir.
- 📌 Oluştuğu durumlar:
  - Referans parametreler
  - Pointer atamaları

```
int *X, *Y;
X = (int *) malloc(sizeof(int));
*X = 5;
Y = X;
*Y = 10;
write(*X);
```

Burada X ve Y:

Farklı isimlerdir

Ama:

Aynı bellek konumunu gösterir

Sonuç:

write(*X) → **10** yazdırır

# Blok

- Modern programlama dillerinin çoğu:
  - blok yapısını (block structure) destekler
- Bu yaklaşım ilk olarak:
  - **ALGOL60** ile ortaya çıkmıştır
- 👉 Blok yapısı:
  - Ortamın (environment) organizasyonu için temel mekanizmadır
- Bloklar sadece bildirim için değil:
  - Komutları gruplayarak tek bir bileşik komut (composite command) gibi davranır

- Block
  - Başlangıç ve bitiş işaretleri ile belirlenen, içinde yerel bildirimler (declarations) barındırabilen program parçası
- Yani:
  - Belirli bir metinsel bölge (textual region)
- Blok sınırları dile göre değişir:
  - ALGOL → begin ... end
  - C / Java → { ... }
  - LISP → ( ... )
  - ML → let ... in ... end

# Blok

- Blok Türleri
  - 1. Prosedür Bloğu
    - Bir prosedüre ait blok
    - Gövde + parametre bildirimleri
  - 2. Satır-içi Blok (In-line Block)
    - Prosedürle ilişkili değildir
    - Komut yazılabilen her yerde bulunabilir.

- **Ortamın Değişimi**
  - Ortam (environment) program çalışırken değişir
  - Ancak bu değişim genellikle:
    - **Bloka girişte**
    - **Bloktan çıkışta** olur
    - 👉 Bu yüzden:
      - Blok = **sabit ortamın en küçük birimi**

# Blok

- Bir blok çalıştığında:
  - Ortam başlangıçta şunları içerir:
    - Yerel bildirimler
    - Çoğu dilde:Bloklar iç içe (nested) olabilir.
- Blok İç İçe Geçme (Nesting)
  - Bir blok:Başka bir bloğun içinde tanımlanabilir
  - ❌ Geçersiz durum:
    - A aç
    - B aç
    - A kapat
    - B kapat
  - ✓ Doğru yapı:LIFO (stack-like) kapanış
- Görünürlük (Visibility)
  - Bir blokta tanımlanan isim o blokta ve İçindeki tüm bloklarda görünürdür.
  - ⚠️ İstisna: aynı isim yeniden tanımlanırsa yeni tanım eskisini gizler (shadowing)
- Shadowing (Gölgeleme)
  - İç blokta aynı isim yeniden tanımlanırsa:
  - 👉 İç blokta:Yeni tanım geçerlidir.
  - 👉 Dış blokta:Eski tanım geçerlidir
  - 👉 İç blok süresince dış tanım devre dışı kalır (visibility hole).

# Blok

```
A:{int a =1;

   B:{int b = 2;
      int c = 2;

      C:{int c =3;
         int d;
         d = a+b+c;
         write(d)
      }

      D:{int e;
         e = a+b+c;
         write(e)
      }
   }
}
```

- Nesne (Object) İşlemleri
  1. Oluşturma (Creation)
     - Bellek tahsisi yapılır
     - (Opsiyonel) başlangıç değeri atanır
  2. Erişim (Access)
     - İsim üzerinden değere ulaşılır
  3. Değiştirme (Modification)Değer güncellenir
  4. Yok Etme (Destruction)Bellek serbest bırakılır

# Blok

- Nesne ömrü ≠ bağlama ömrü olabilir.
- Referans ile parametre geçirme
- 👉 Nesne: Daha uzun yaşar
- 👉 Bağlama: Daha kısa sürede yok olur

- Dangling Reference (Sarkan Referans)
- Bir isim:
  - Artık var olmayan nesneye erişirse
- Bu duruma Dangling reference denir.
- 📌 Sebep: Bellek serbest bırakılmıştır
- ⚠️ Bu bir hata durumudur

# Statik Skop

- Scope Problemi (Motivasyon)
- Aşağıdaki program:
- A:{
  int x = 0;
  void fie(){
    x = 1;
  }
  B:{
    int x;
    fie();
  }
  write(x);
}
- 👉 Soru: write(x) ne yazdırır?

- Scope Türleri
  - Static (Lexical) Scope
    - Metin yapısına bağlı
  - Dynamic Scope
    - Çalışma akışına bağlı
- Statik Scope Kuralı
  - En Yakın Kapsam (Nearest Scope Rule)
  - Önce yerel blok aranır, yoksa dış bloklara gidilir.
  - En dış blok → global → predefined

# Statik Skop

- Statik Scope Sınırlamaları
- Derleyici:
  - Bellek adresini ve değeri tam bilemez
- Ancak:
  - Offset bilgisini bilir → hızlı erişim

- Statik Scope Avantajları
  - Program okunabilirliği ↑
  - Derleyici analiz yapabilir
  - Tür kontrolü mümkün
  - Optimizasyon yapılabilir
- Çoğu dil statik skop kullanır
- 📌 Örnekler: C, C++, Java, Ada, Pascal, Scheme

# Dinamik Skop

- Ortam:
  - Dinamik olarak değişir
  - Scope kuralları ile belirlenir.
  - Statik scope: Öngörülebilir ve verimlidir.
  - Dinamik scope:Daha esnek ama karmaşıktır.

- Bazı dillerde dinamik skop kullanılır:APL, LISP (bazı versiyonlar), SNOBOL, PERL
- Amaç: runtime ortam yönetimini basitleştirmek
- Statik scope, runtime’da karmaşık bir yönetim gerektirir:Non-local ortamlar, blokların normal aktivasyon/deaktivasyon sırasını yansıtmaz.

# Dinamik Skop

- Statik Scope ve Blok Sırası
- Non-local isim çözümü:Blokların açılma/çıkma sırasından farklıdır.
- Runtime’da stack (LIFO) kullanılır
- Derleyici için:Söz dizimi (syntactic structure) üzerinden çözümleme yapılır

```
const x = 0;
void fie() { write(x); }
void foo() { const x = 1; foo(); }
foo();
```

Dynamic scope kuralı:En son oluşturulmuş, halen aktif bağlama kullanılır.
Runtime’da:Non-local isim çözümü için yalnızca stack yeterlidir.
Bu durumda yukarıdaki örnek 1 yazar.

# Dinamik Skop

- Bir isim X’in geçerli bağlaması, programın herhangi bir P noktasına ulaşıldığında, X için en son oluşturulmuş ve hâlâ aktif olan bağlamadır.
- Local ve global environment:
  - Statik ve dinamik scope’da aynı
  - Non-local environment:Statik: metin yapısına bağlı
  - Dinamik: çalıştırma sırasına bağlı

# Özet

- 1. Denotable Objects (İsim Verilebilen Nesneler)
  - İsim atanabilen tüm nesneler: değişkenler, sabitler, prosedürler, türler
  - 2. Environment (Ortam / Çevre)Runtime'da isim-nesne ilişkilerini tutar: Local, Non-local, Global environment olarak üçe ayrılır. Yüksek seviyeli dillerin temel özelliği
  - 3. Bloklar (Blocks)Programın metinsel bölgeleri, yerel deklarasyonları içerir. Görünürlük ve environment yönetimi için temel yapı
  - 4. Environment Üzerindeki Operasyonlar
    - İsim-nesne: oluşturma, referans, devre dışı bırakma, tekrar etkinleştirme, yok etme
    - Nesne: oluşturma, erişim, değiştirme, yok etme
    - Nesne ömrü ile isim-nesne bağlaması ömrü her zaman eşleşmez
- 5. Scope Kuralları (Görünürlük)
  - Static (Lexical) Scope: Compile-time, program metni esas alınır, modern dillerde yaygın
  - Dynamic Scope: Runtime, son aktif bağlam esas alınır, nadiren kullanılırKurallar, bloklar arası isim erişimini belirler
- Environment + Scope = geçerli isim-nesne ilişkisi
- Bloklar ve görünürlük kuralları program davranışını doğrudan etkiler
- Static scope: okunabilir, optimize edilebilir
- Dynamic scope: esnek ama karmaşık ve verimsiz